INSTRUCCIONES CARGA Y ALMACENAMIENTO

viernes, 11 de septiembre de 2026 07:16 a. m.

Sesión 7

UII. 2.2.1
Operandos en
- **Registros:**

**Procesador
**Acceso rápido
**Cantidad limitada

- **Memoria:**

**Fuera de registros
**Mayor capacidad
**Acceso lento

Load word } Acceden a memoria
Store word } de datos
ALU } con registros

FLUJO
Memoria -> LOAD -> Registro -> Operación -> Registro -> STORE -> Memoria

lw $t0, 0($s0) **-----> se carga un dato en t0**
lw $t1, 4($s0) **-----> se carga un dato en t1**
add $t2, $t0, $t1 **-----> se suman los registros**
sw $t2, 8($s0) **-----> el resultado se almacena en memoria**

>>> lw: load word
- Carga una palabra de memoria hacia un registro
  lw $t0, offset($s0)

>>> sw: store word
- Almacena una palabra desde un registro hacia la memoria sw$t0, offset($s0)

LOAD: Memoria ---> Registro
SOTORE: Registro ---> Memoria

Instrucción load word.
Su función es leer una palabra de memoria y colocarla en un registro

sintaxis
lw rt, offset(base)
lw $t0, 8($s0)

Campo	Componente
Operación	lw
Registro destino	$t0
Offset	8
Registro base	$s0

8($s0) -> Indica dirección de memoria

Desplazamiento

Contenido del registro base

Por lo tanto lw $t0, 8($s0) significa **desplazamiento** y **dirección base**

Memoria[$s0 + 8] -> $t0

suponer: $s0 = 1000
lw $t0, 12($s0)

Identificar la base: $s0
Identificar offset: 12
Calcular dirección efectiva: 1000 + 12 = 1012
Leer memoria: memoria[1012] -> $t0

si memoria[1012] = 25
$t0 = 25

---

suponer: $s0 = 1000 y una memoria:

lw $t0, 8($s0)
1000 + 8 = 1008
Memoria[1008] -> $t0
$t0 = 38

Dirección	Contenido
1000	15
1004	27
1008	38
1012	53

Memoria direccionable por byte. El offset se expresa en bytes.

Suponer: 1000
1004
1008
1012

Considerar:	Dirección
lw $t0, 0($s0)	1000
lw $t1, 4($s0)	1004
lw $t2, 8($s0)	1008
lw $t3, 12($s0)	1012

si $s0 = 1000

En un arreglo <--
Suponer A[4] = {10, 20, 30, 40}
cada elemento es una palabra de 32bits
A[0] = 4 byte
A[1] = 4 byte
A[2] = 4 byte
A[3] = 4 byte

Si $s0 contiene la dirección inicial de A

		Resultado
lw $t0, 0($s0)	A[0]	$t0 = 10
lw $t1, 4($s0)	A[1]	$t1 = 20
lw $t2, 8($s0)	A[2]	$t2 = 30
lw $t3, 12($s0)	A[3}	$t3 = 40

Store word
Transfiere una palabra registro a memoria
Sintaxis
Sw, rt, offset (base)

sw $t0, 8($s0)
significa
Memoria [$s0 + 8] <- $t0

Suponer
$s0 = 2000
$t0 = 75
sw $t0, 8($s0)

Registro base: $s0
Offset: 8
Dirección efectiva: 2008

Almacenar
memoria[2008] <- $t0
memoria[2008] <- 75

Diferencias

Caracteristica	lw (load word)	sw (store word)
Lee en memoria	Si	No
Escribe en memoria	No	Si
Dirección	Base + offset	Base + offset
Transferencia	Memoria -> Registro	Registro -> Memoria

Que formato utilizan?
- Tipo inmediato (I)

Op	Rs	Rt	Imm
	Registro base	Registro que recibe o proporciona el dato	
*	$s1	$t1	4

<---------- Formato de la operación

Lw $t0, 4($s1) <--------- sintaxis en la operación

sw $t0, 8($s0) ----------> rs: $s0. rt: $t0. imm: 8.

Se requiere

C = A + B

Suponer el almacenamiento siguiente:

Dirección	Variable	Dato	Y $S0 = 1000
1000	A	20	
1004	B	30	
1008	C		

Desplazamiento implícito en la dirección

PASOS 1	**PASO 2**	**PASO 3**	**PASO 4**
Cargar A	Cargar B	Sumar	Guardar
lw $t0, 0($s0)	lw $t1, 4($s0)	add $t2, $t1, $t0	sw $t2, 8($s0)

$t2: variable donde se guarda el resultado de la suma

Suponiendo
$s0 = 1000
$t0 = A
$t1 = B
$t2 = 50

El offset puede ser negativo
lw $t0, -4($s0)
con $s0 = 1000
Dirección efectiva: 1000 + (-4) = 996
Memoria[996] -> $t0

suponer
$s0 = 2000

Determinar $t0, $t1 y $t2 y	Dirección	Contenido
	2000	8
	2004	16
	2008	24

lw $t0, 0($s0)
lw $t1, 4($s0)
lw $t2, 8($s0)

	Offset	Dirección efectiva	base	$t0
lw $t0, 0($s0)	0	2000	2000	8
lw $t1, 4($s0)	4	2004	2000	16
lw $t2, 8($s0)	8	2008	2000	24

Suponer:

Suponer:

$s0 = 3000
$t0 = 100
$t1 = 250

memoria	dato	**nuevo dato |**
3000	10	
3004	20	100
3008	30	250

Ejecutar

1. sw $t0, 4($s0)
2. sw $t1, 8($s0)

1, 2 BASE	$s0 = 3000
1 offset	4
2	8

****como es store
se guarda del registro
a la memoria*****

Para

1.

Memoria efectiva

3000 + 4 = 3004

Memoria [3004] <- $t0

Memoria[3004] <- 100

2.

Memoria efectiva

3000 + 8 = 3008

Memoria [3008] <- $t1

Memoria [3008] <- 250